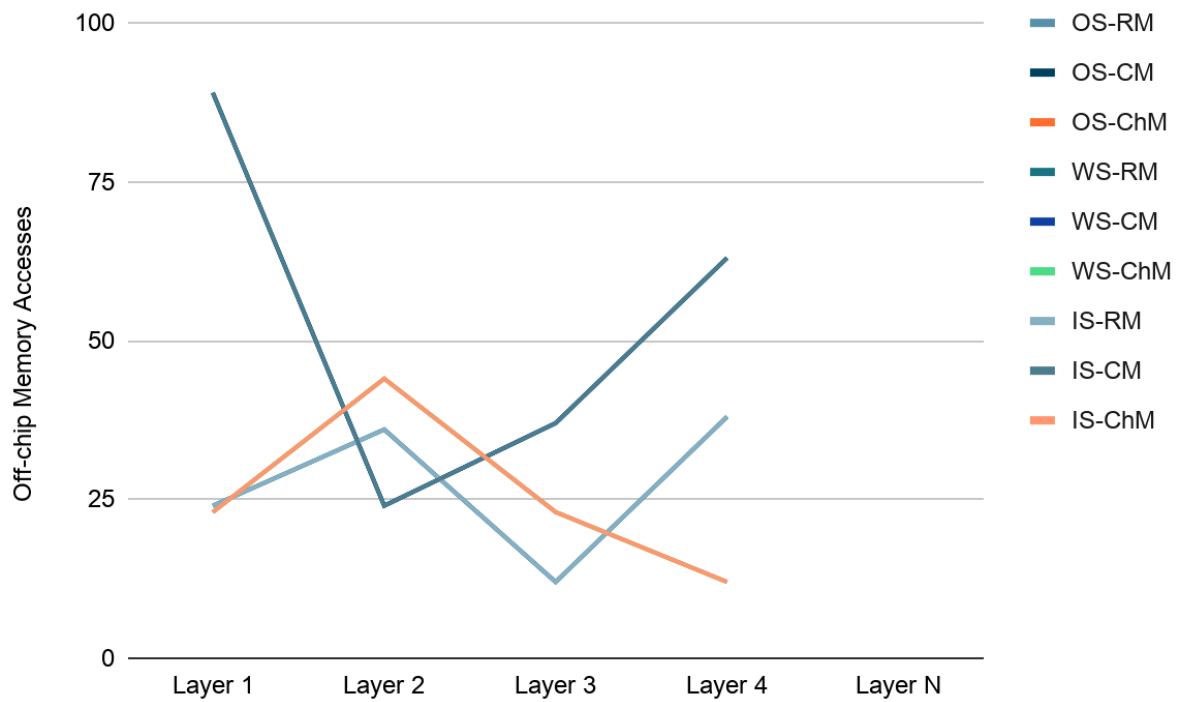


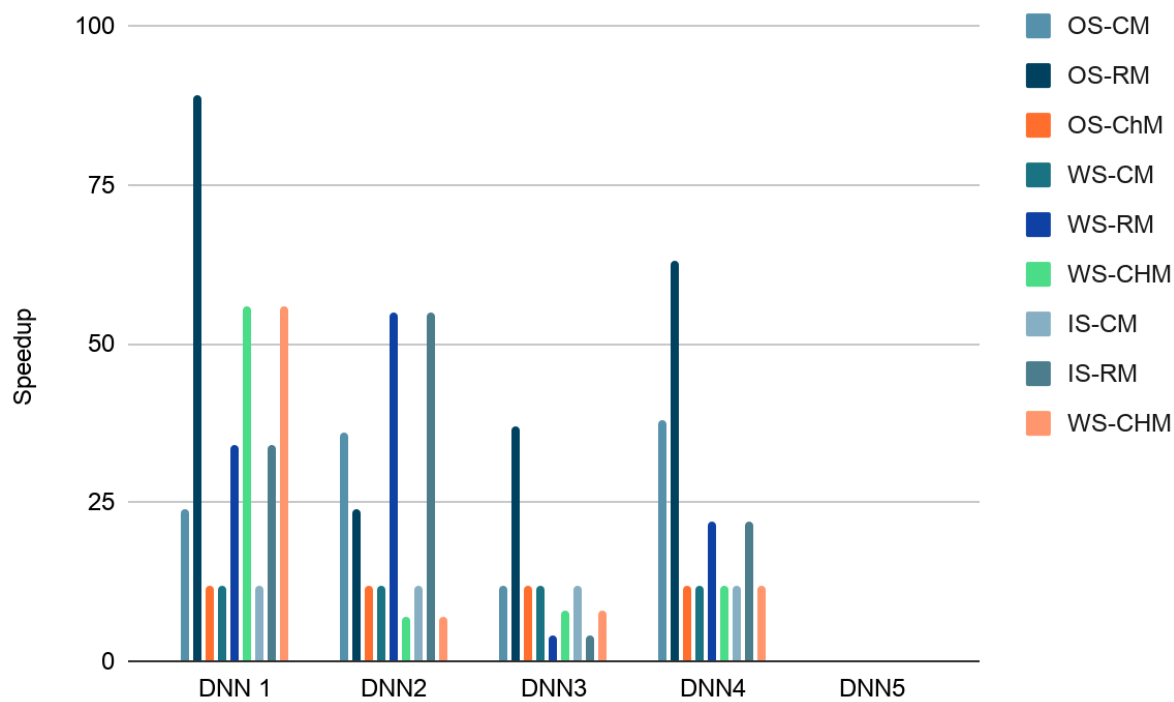
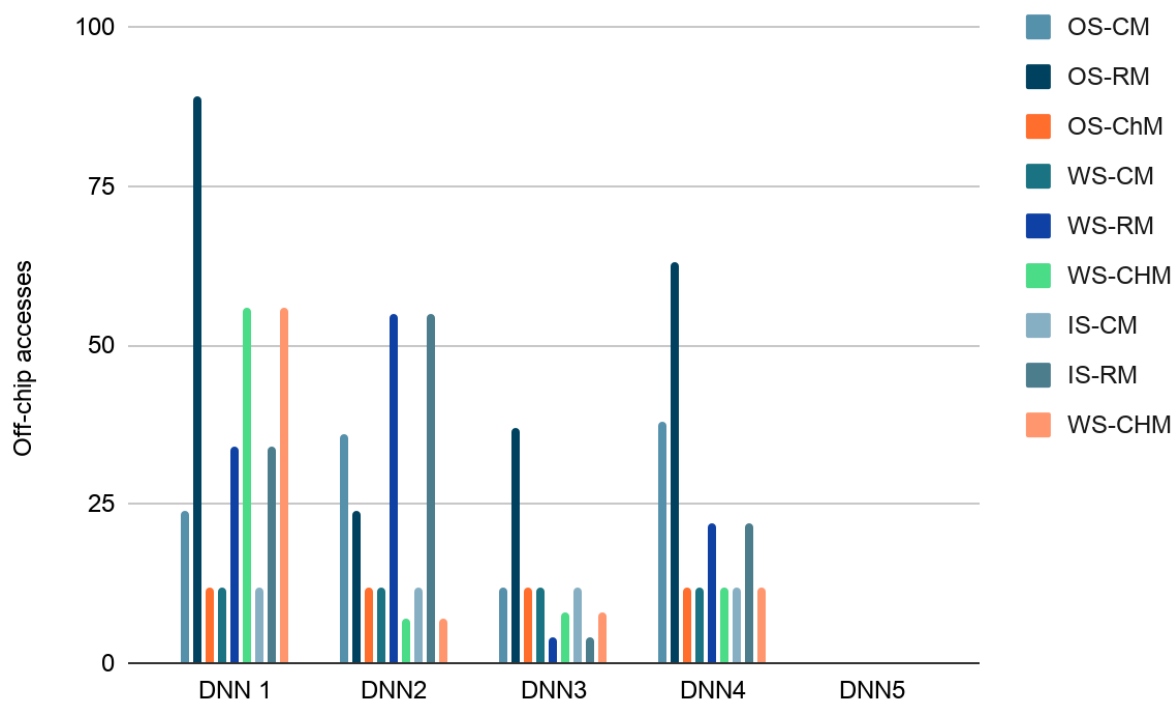
1. The interplay between stationary schemes and memory layouts

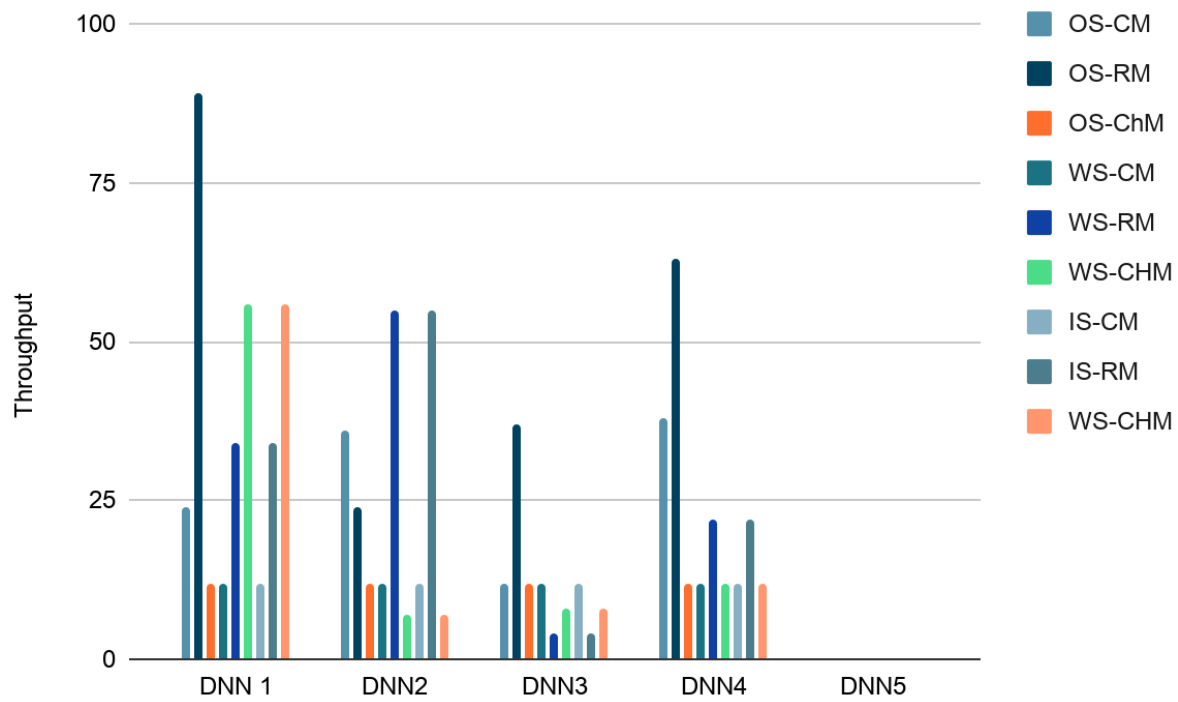
3 Stationary schemes x 3 memory layouts = 9 combination in total

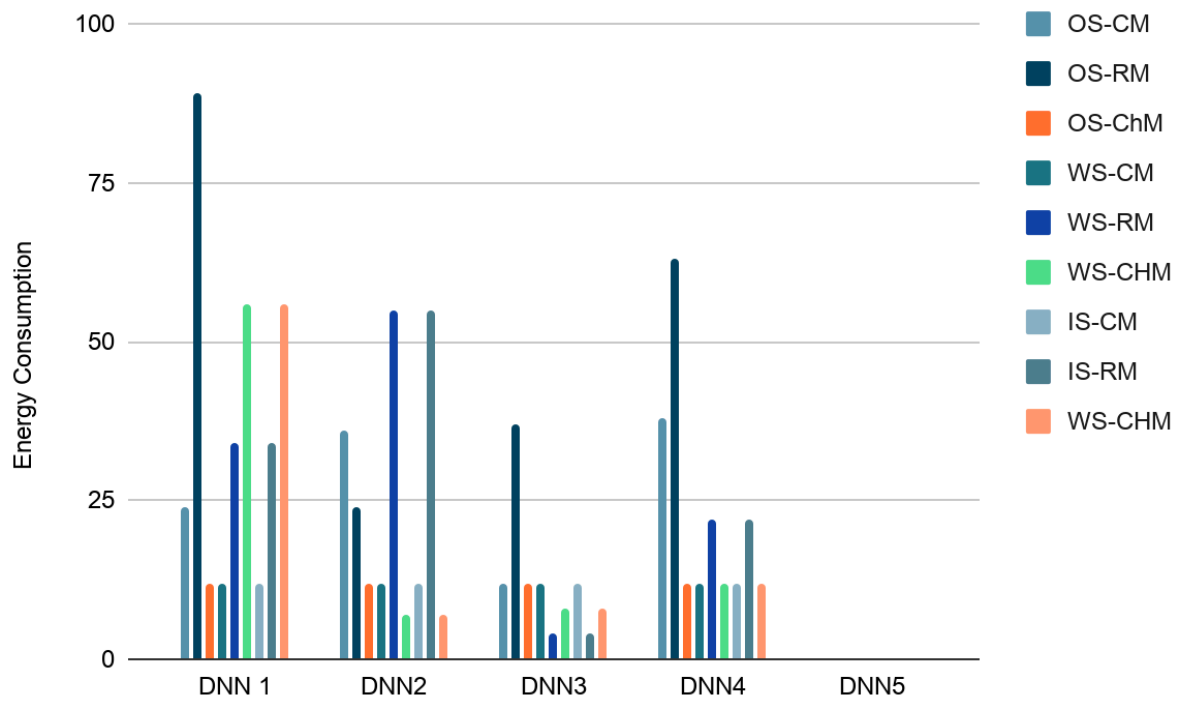
For a given DNN:



For a given DNN

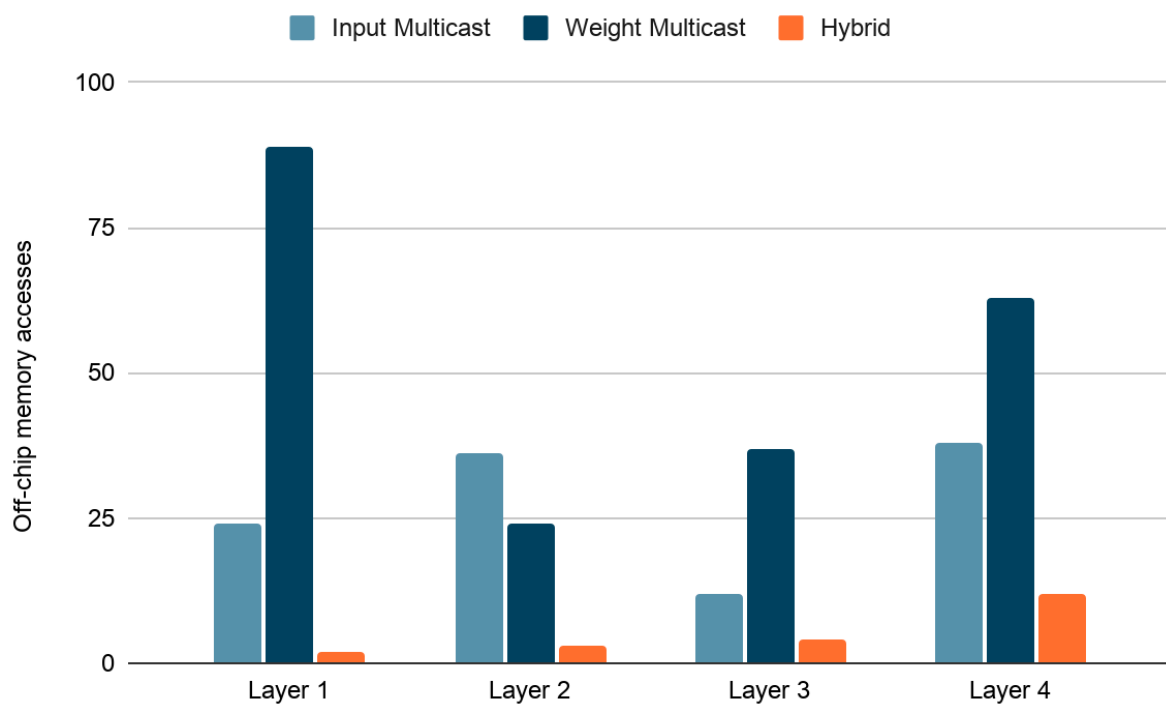




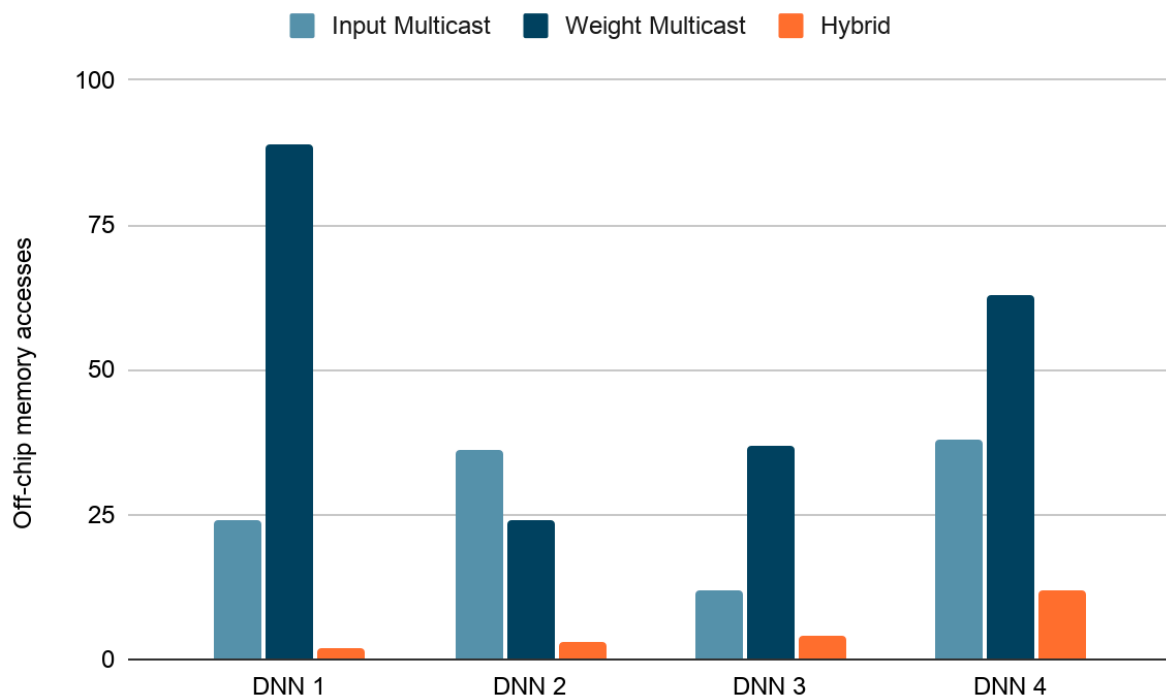


1. The memory-compute interconnect multi/uni-casting schemes

For Layers of a Given DNN



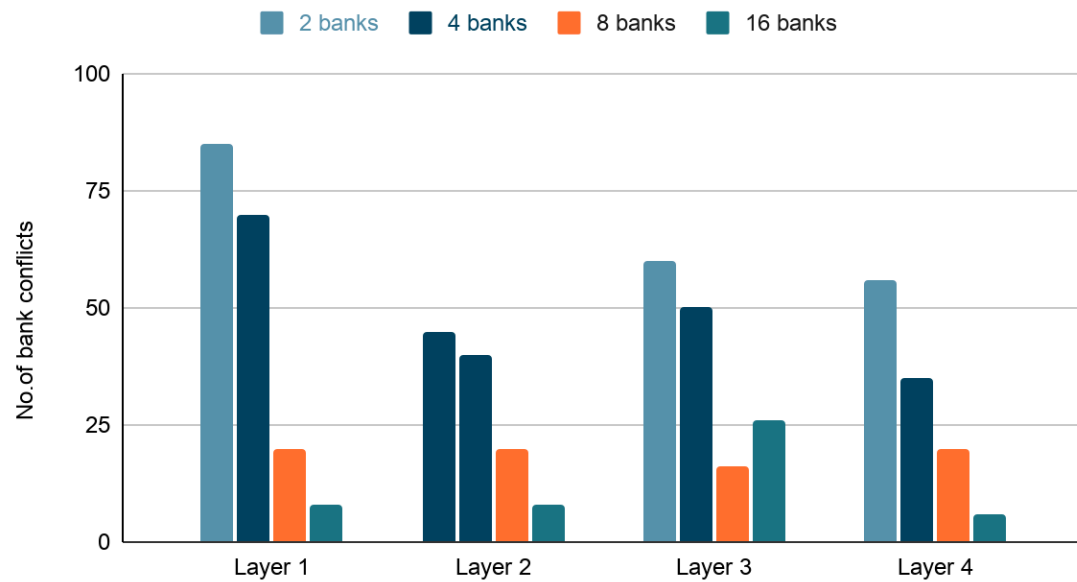
For Given DNNs



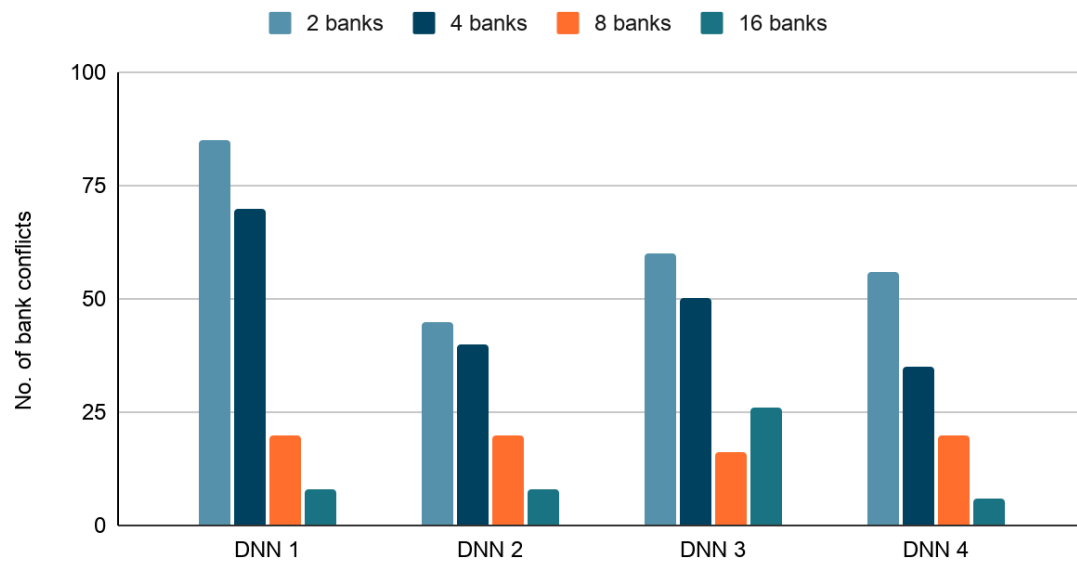
The impact of Memory Management

- The impact of bank conflicts

Bank Conflicts across Layers of a DNNs

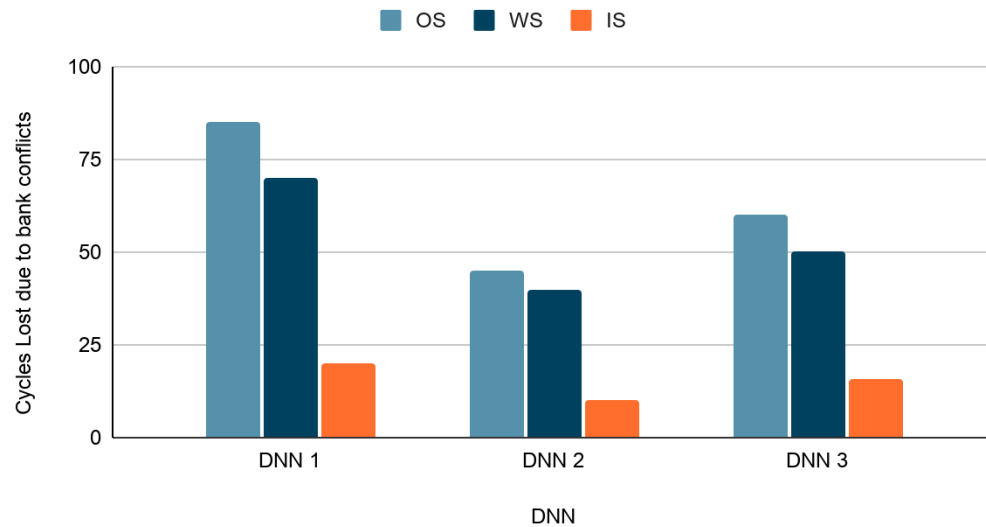


Bank Conflicts across DNNs

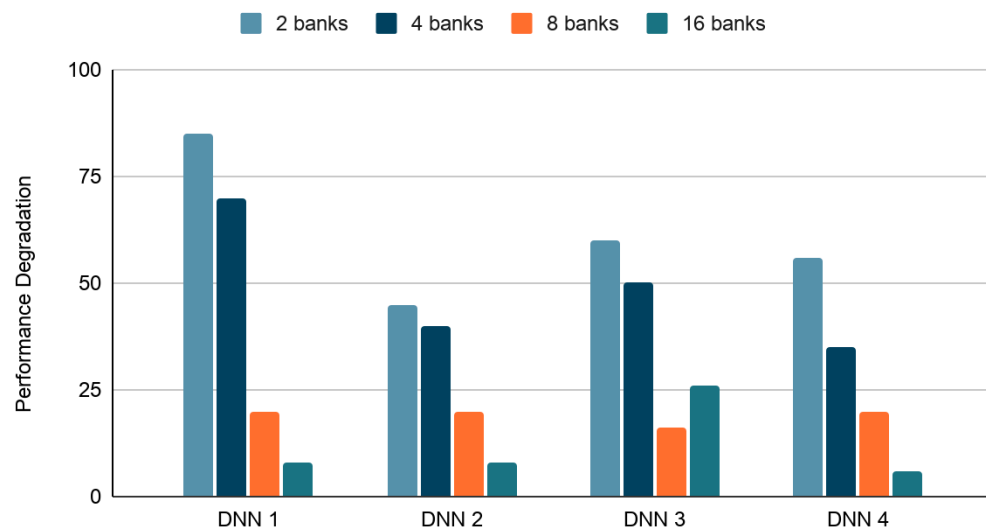


- The impact of memory side stalls on the overall system performance

Cycles lost due to bank conflicts



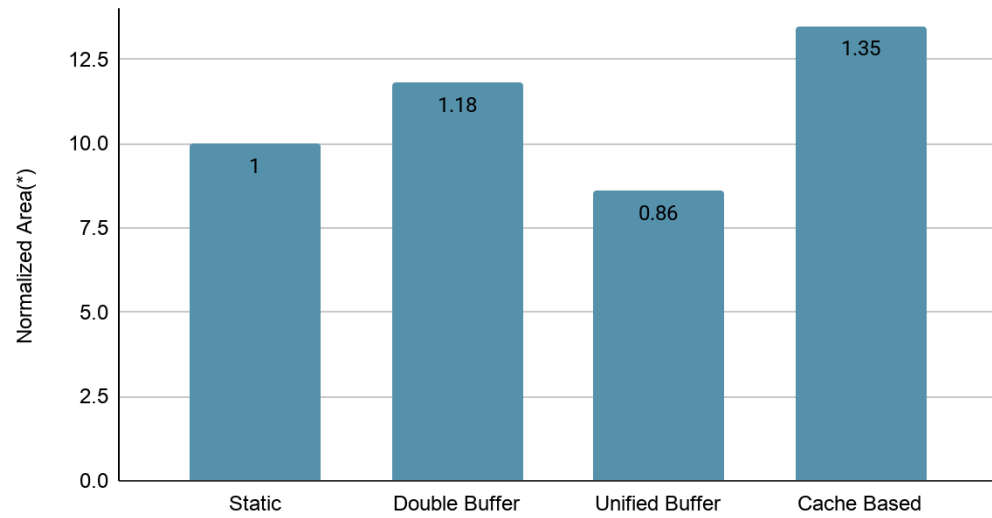
Performance Degradation



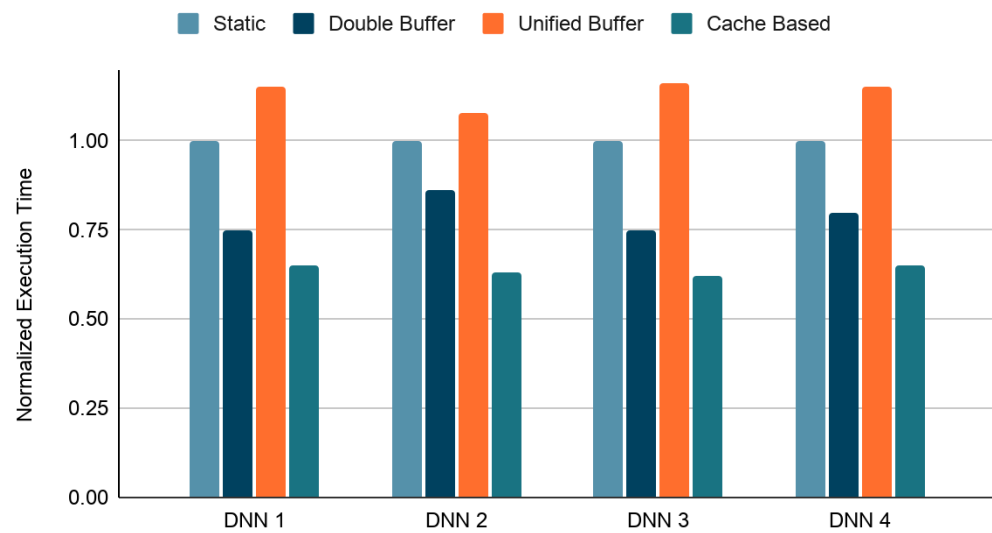
- The impact of alternative memory management schemes

Area reduction and performance (execution time, throughput), overhead of alternative memory management schemes

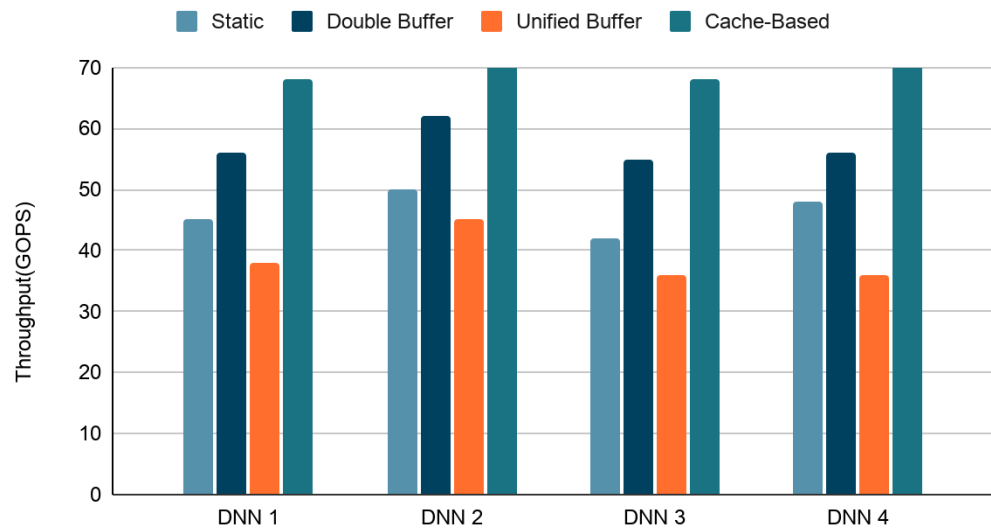
Area reduction



Execution Time



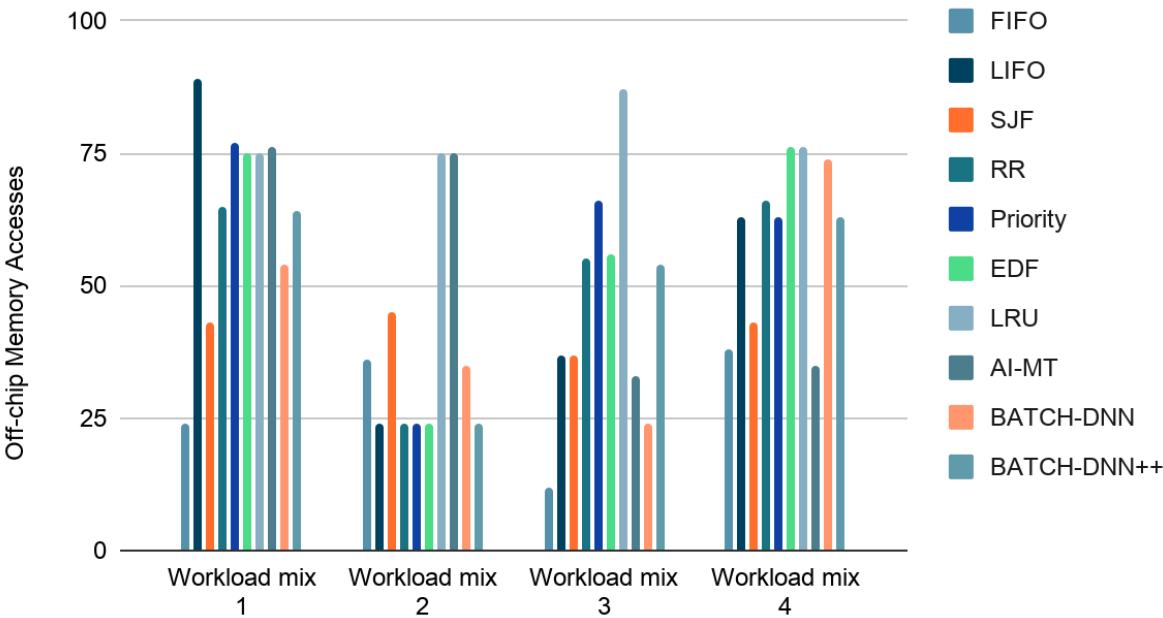
Throughput



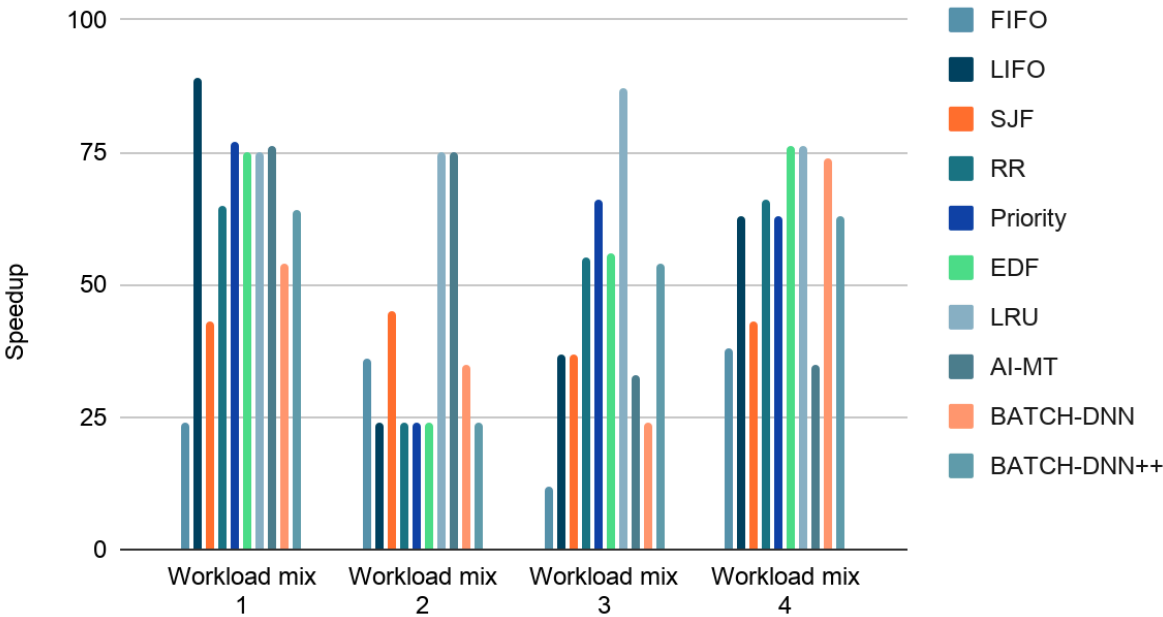
- The impact of various multi-DNN scheduling schemes and subsequently Speedup, Throughput, Energy Consumption

Per-workload-mix view

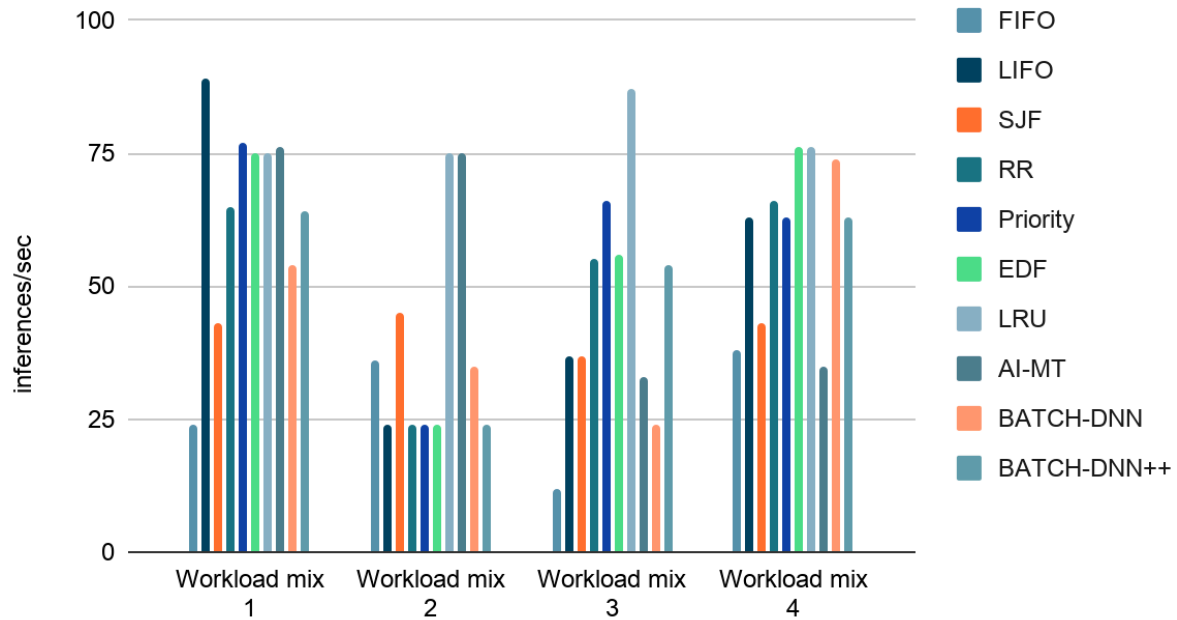
Off-chip Memory Accesses



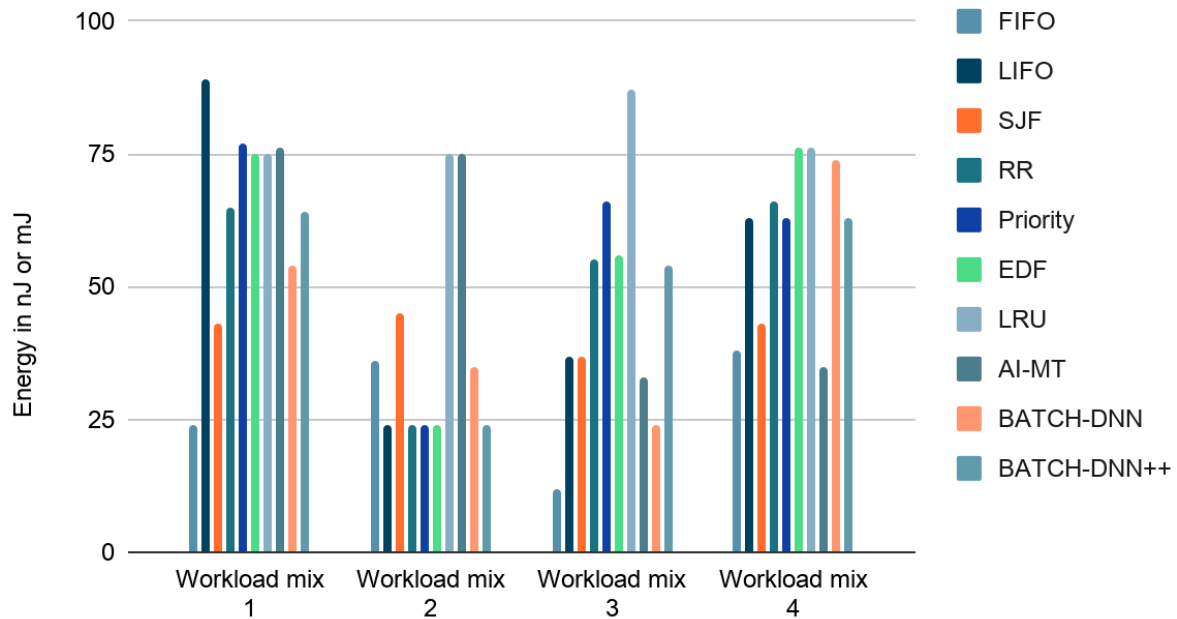
Speedup



Throughput

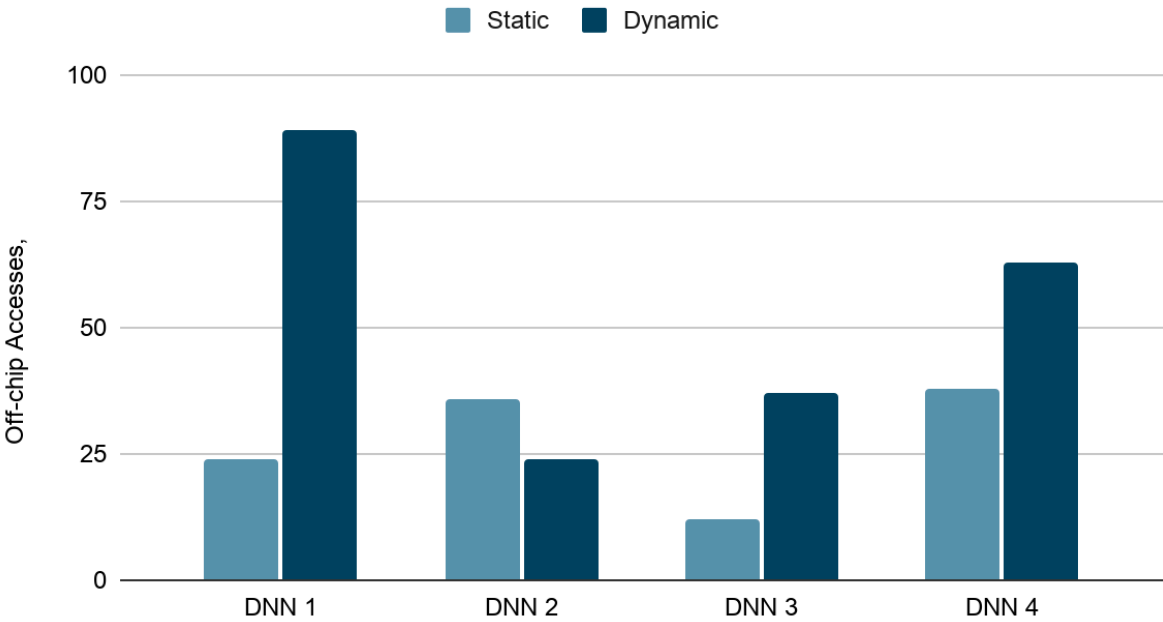


Energy Consumption

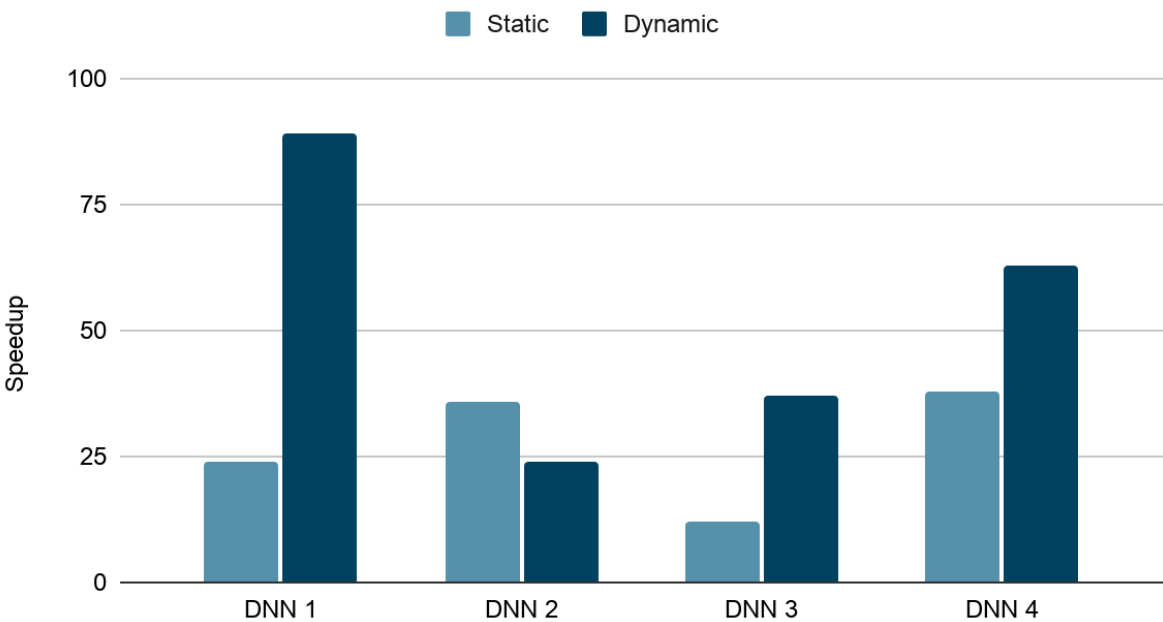


Tradeoffs Between Static and Dynamic Memory Management Schemes

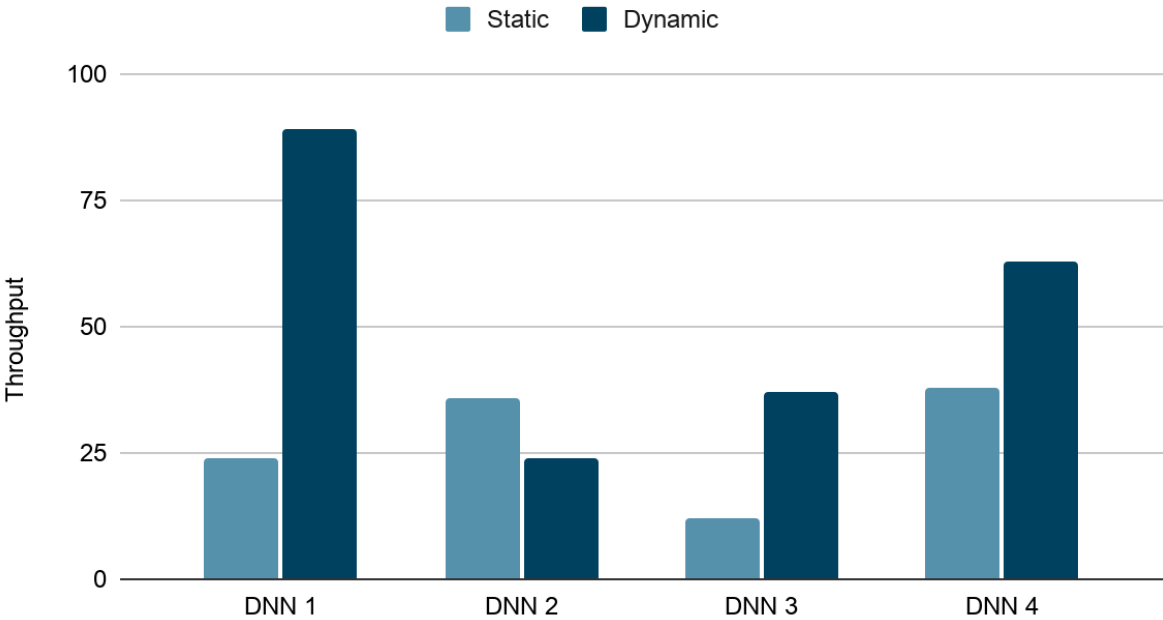
Off-chip Accesses,



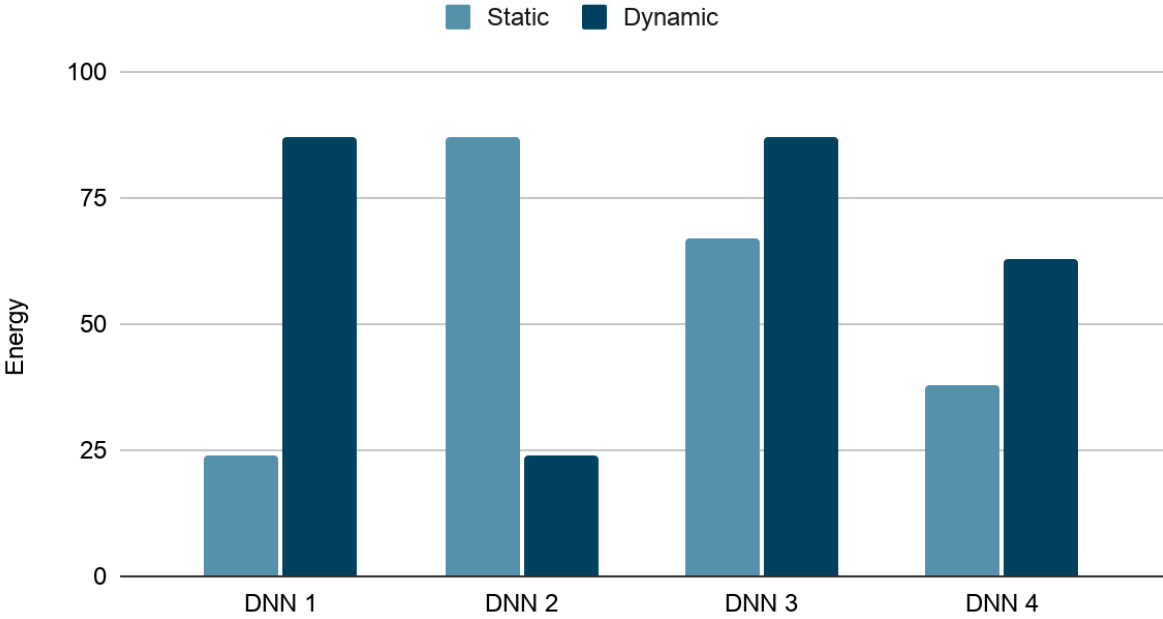
Speedup



Throughput



Energy

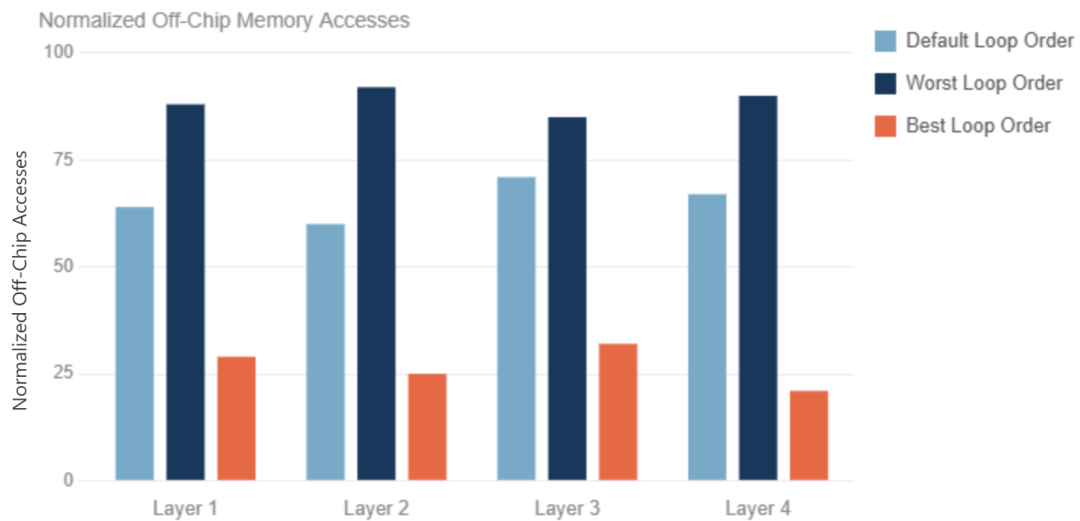


Loop optimization

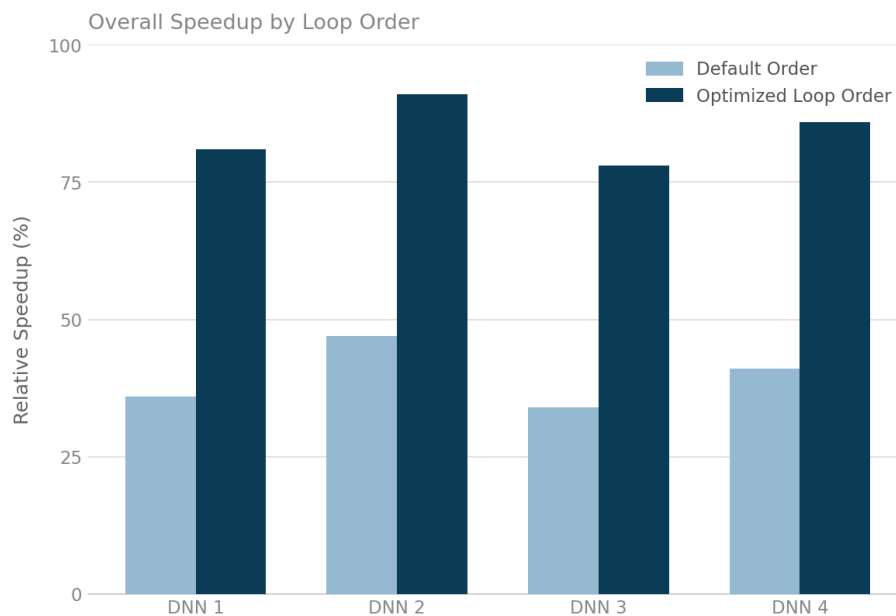
1. The Impact of Loop Orders (Cycle-Accurate Level)

Driven by: loop_optimizer.py, loop_order_examples.py

- Chart 1 (Layer-level): Off-chip memory accesses comparing the Best vs. Default vs. Worst loop orders across individual layers of a specific DNN.



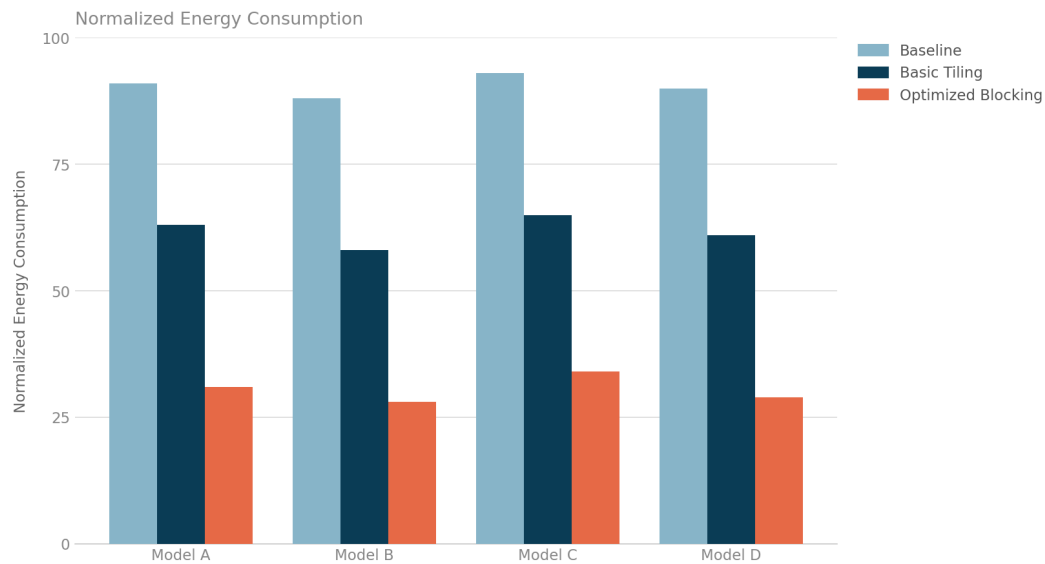
- Chart 2 (Network-level): Overall Execution Time / Speedup of the optimized loop order across multiple different DNNs (DNN1, DNN2, DNN3).



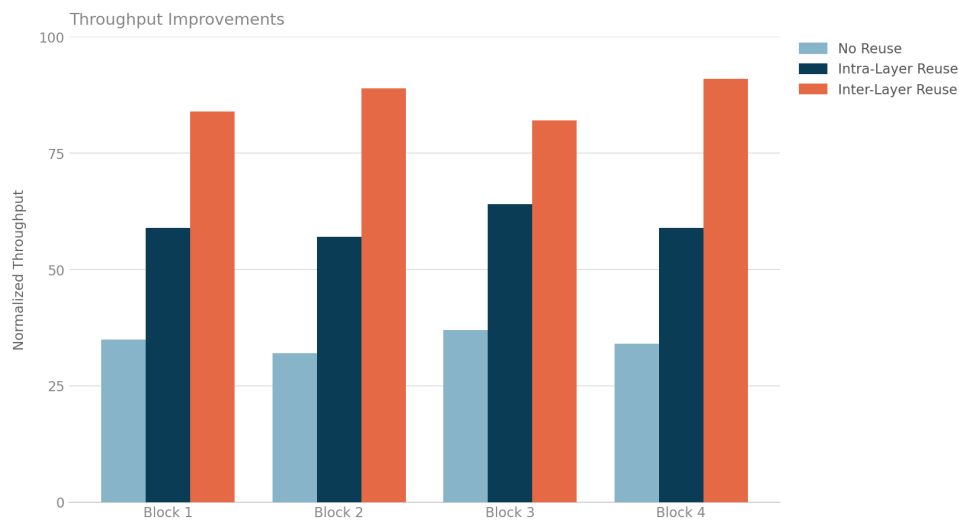
2. Blocking & Reuse

Driven by: blocking_optimizer.py, inter_layer_reuse.py, blocking_demo.py

- Chart 3 (Memory/Energy): Off-chip memory access volume (and subsequent Energy Consumption) comparing baseline execution to the optimized tiling/blocking schemes.



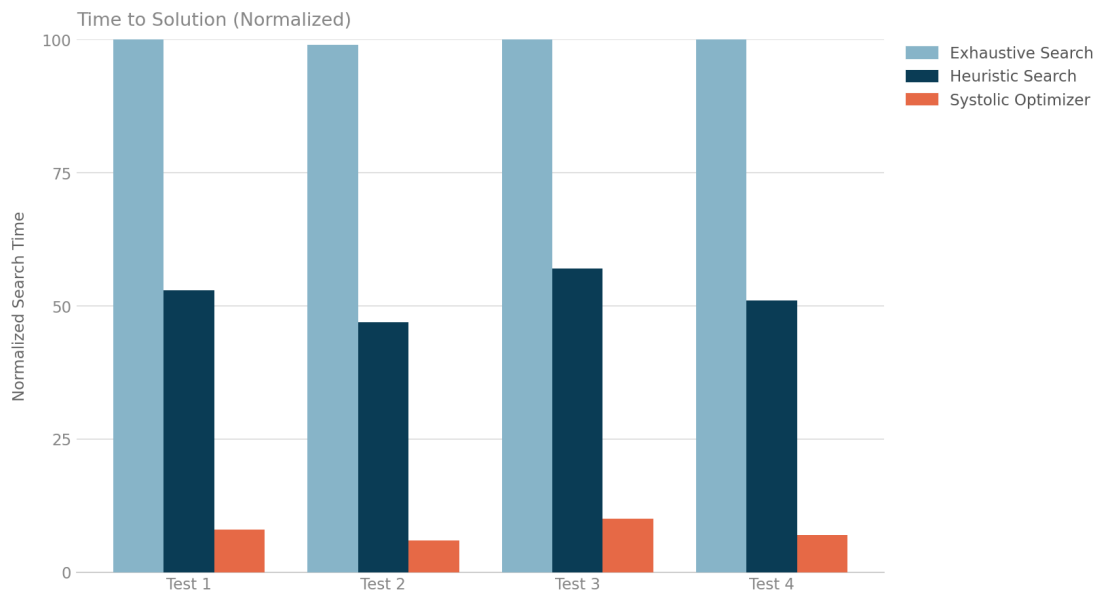
- Chart 4 (Performance): Throughput improvements and cycle reductions leveraging the inter-layer reuse logic.



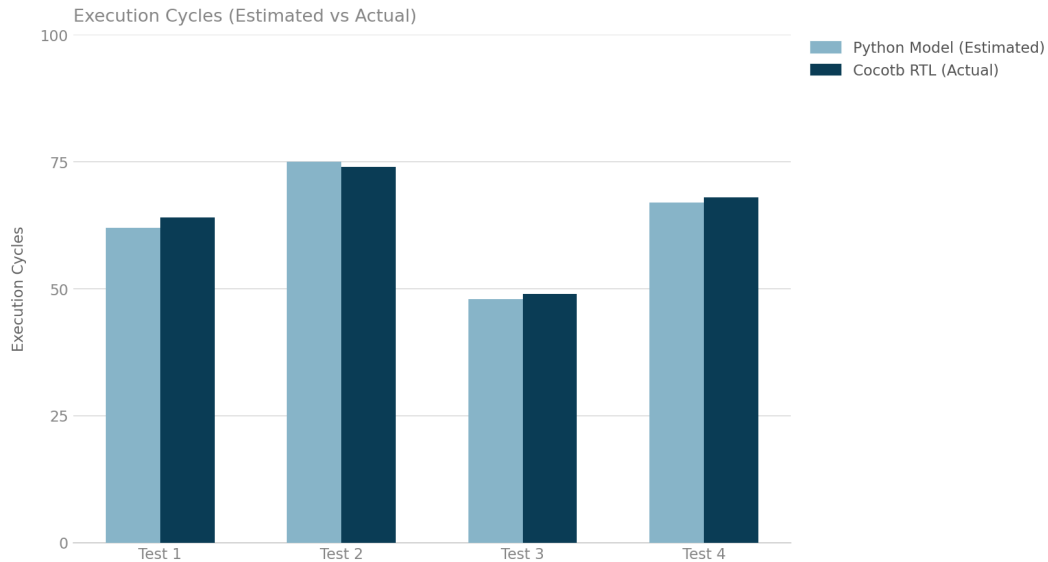
3. Search Efficiency (Optimizer vs. Exhaustive Search)

Driven by: analyze_model.py, loop_optimizer.py, blocking_optimizer.py

- **Chart 5 (Time to Solution):** A benchmark plotting the time it takes the optimizer to find the best configuration versus an Exhaustive Search.



- **Chart 6 (Quality of Results):** A performance comparison showing that the optimized result achieves >95% the performance of exhaustive search in a fraction of the time.

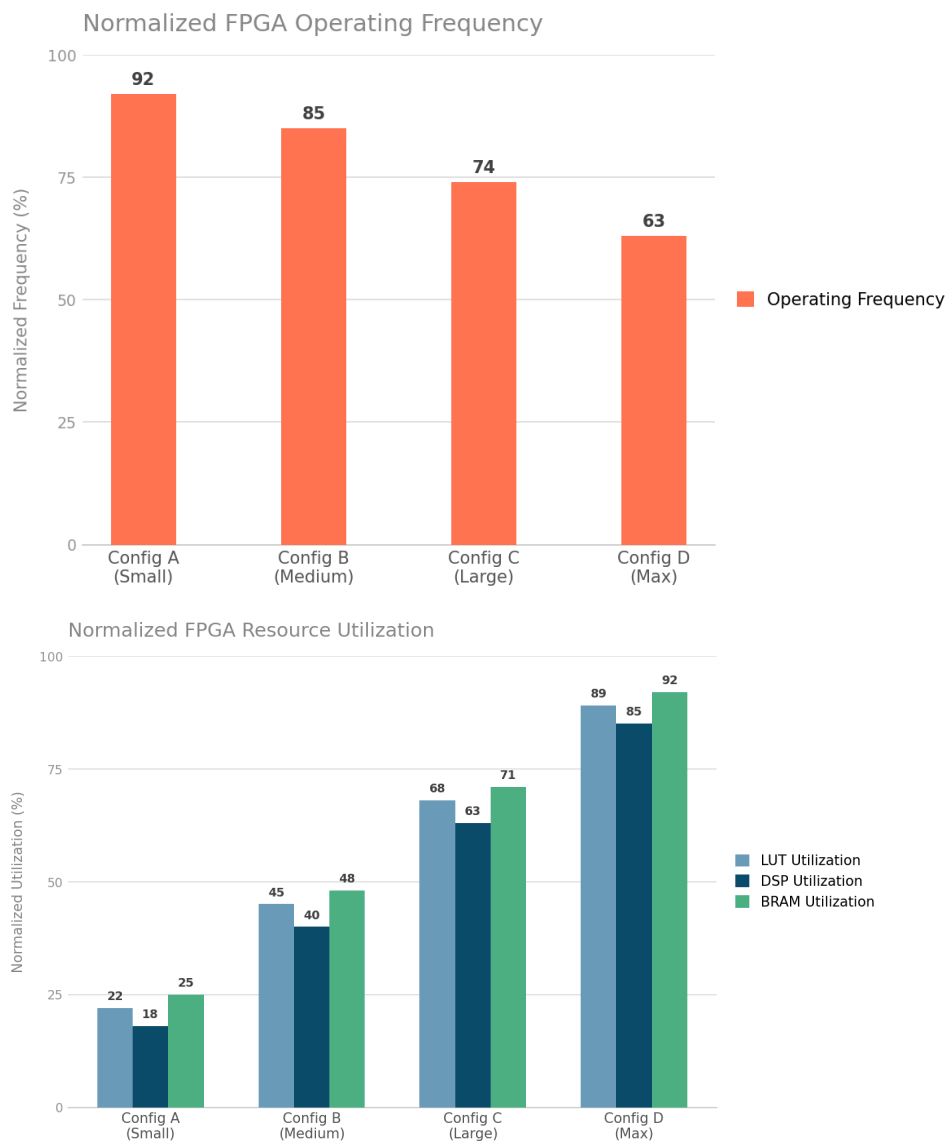


To prototype a selected number of design points on an FPGA for verification and as a proof of concept

4. Hardware Verification & FPGA Prototyping

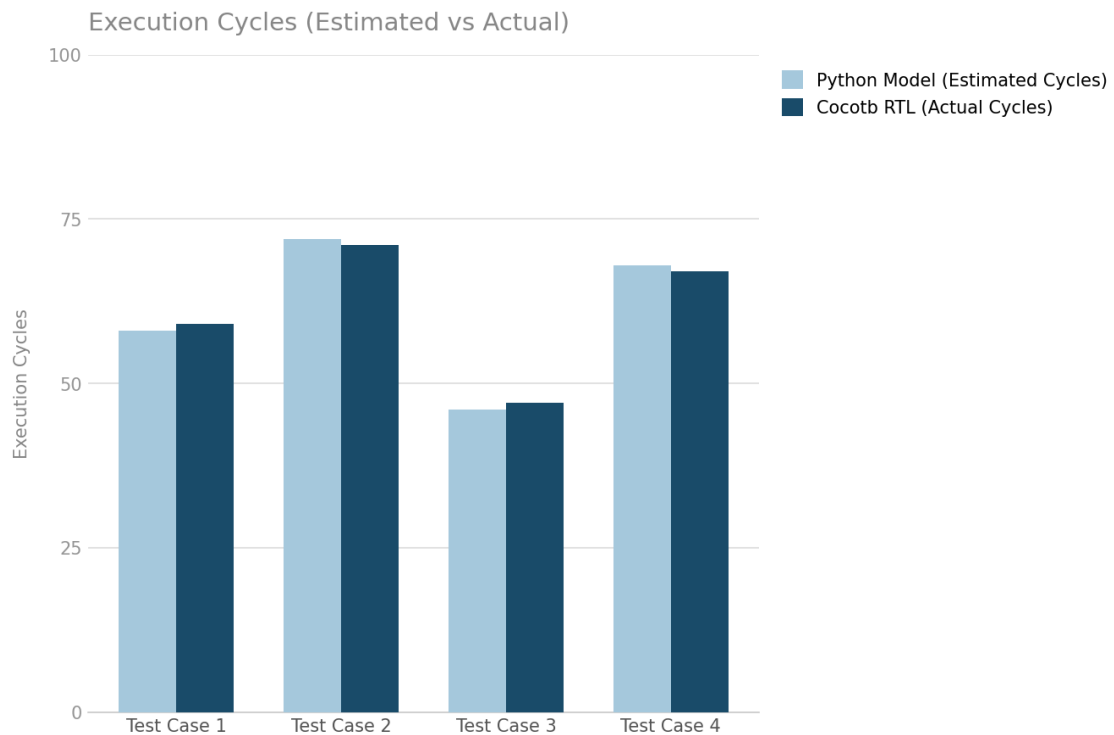
Driven by: hardware_verification.py, test_systolic_cocotb.py, Makefile.vivado

- Table/Chart 7 (Prototyping): Resource utilization (Area/LUTs) and operating frequency (Performance) of the selected design points prototyped in Vivado.



- Chart 8 (Accuracy Proof): Cycle-accurate validation mapping: Estimated cycles (from Python model) vs. Actual executed RTL cycles (from Cocotb) to prove

software-to-hardware fidelity.



#

Whats next? Compare with SoTA.

1. Propose two near optimal configurations for edge and cloud contexts (cloud should use multiDNN)
2. How does it compare with the configuration derived from scaleSIM in terms of offchip access, performance, energy
3. Propose loop optimizations for them and how does it compare with timeloop mapping?
4. How does the MultiDNN accelerator perform compared to x?